Formation Kubernetes

Session 2 — Deployments, Services et le modèle réseau

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session

Deployment en détail

Rolling updates, rollbacks, strategies, ReplicaSet

15 min

Services et réseau

ClusterIP, NodePort, LoadBalancer, DNS interne

15 min

ConfigMap

Externaliser la configuration de vos applications

15 min

Travaux pratiques

Application multi-services : API + Frontend + Services

1h10

Rappel — Session 1

Pod

Plus petite unité déployable
Contient un ou plusieurs containers

Deployment

Gère les réplicas et les mises à jour
Assure le desired state

Service

IP stable + DNS
Expose les pods au réseau

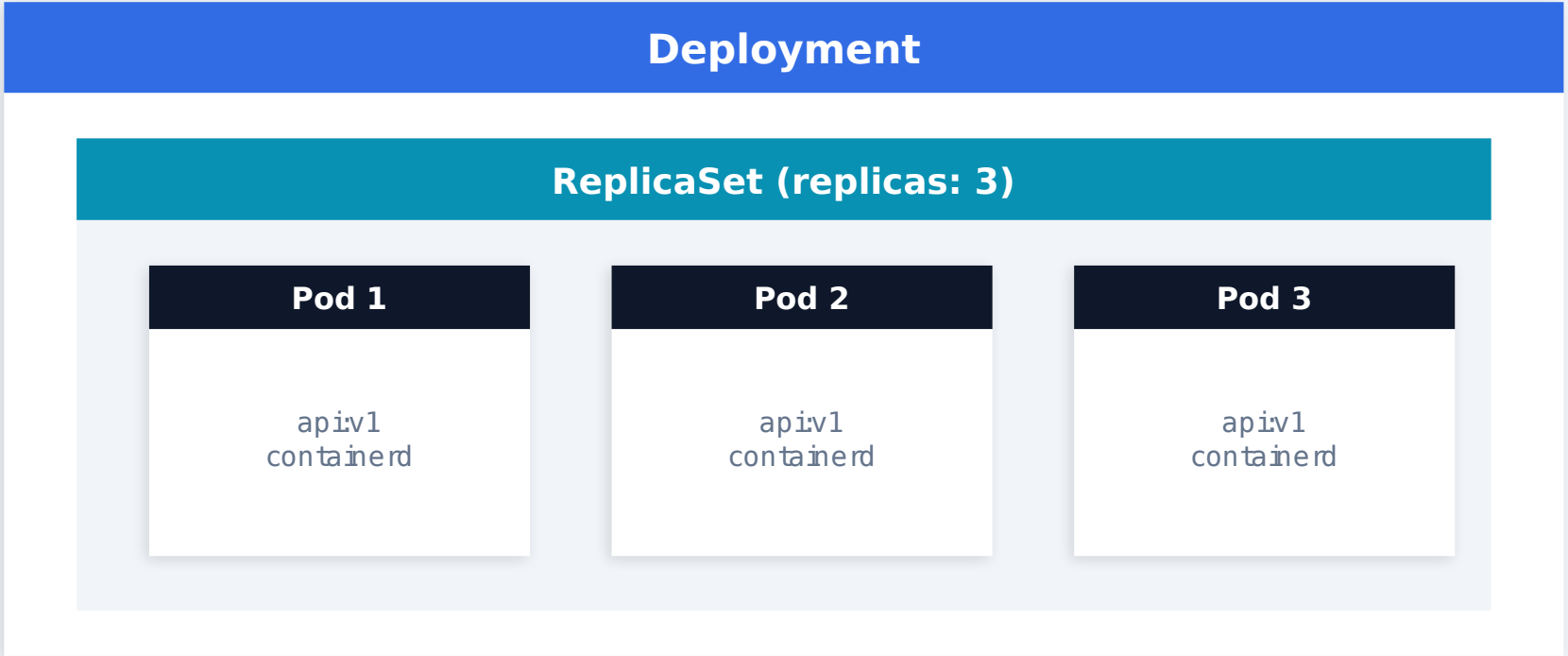
kubectl

CLI pour interagir avec le cluster
apply, get, describe, logs

Aujourd'hui : on approfondit les Deployments et les Services

Deployment en détail

Ce qui tourne sous le capot



Rolling Update & Rollback

Rolling Update

1. Nouveau ReplicaSet créé (v2)
2. Pod v2 démarre et passe les probes
3. Pod v1 est arrêté proprement
4. Répéter jusqu'à 0 pods v1
5. **Zéro downtime !**

Rollback

```
kubectl rollout history
```

Voir les révisions

```
kubectl rollout undo
```

Revenir à la version N-1

```
kubectl rollout undo
  --to-revision=2
```

Vers une révision précise

Services Kubernetes

Une IP stable pour accéder à vos pods

Le problème

Les pods sont éphémères. Leur IP change à chaque redémarrage.

Impossible de hardcoder une IP de pod pour communiquer !

La solution : Service

IP stable + nom DNS.
Load-balancing automatique vers les pods qui matchent le selector.

ClusterIP

défaut

Interne uniquement.
Utilisé pour la communication entre services.

NodePort

debug

Ouvre un port (30000-32767) sur chaque node du cluster.
Rarement utilisé en prod.

LoadBalancer

cloud

Crée un LB cloud (GCP, AWS).
Coûteux si beaucoup de services.
Ingress est préféré.

Service Discovery — DNS interne

Format DNS d'un Service Kubernetes :

`<service>.<namespace>.svc.cluster.local`

Exemples concrets

`api.exercices.svc.cluster.local`	Service 'api' dans le namespace 'exercices'
`api.exercices`	Forme courte (même cluster)
`api`	Forme minimale (même namespace)

ConfigMap

Externaliser la configuration de vos applications

configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
data:
  API_PORT: "8080"
  DB_HOST: "postgres"
  LOG_LEVEL: "info"
```

Utilisation

Variables d'environnement

envFrom: configMapRef
Injecté dans les containers

Fichiers montés

Volume mount dans le pod
Idéal pour nginx.conf, etc.

Arguments CLI

Passé via les args
du container

ConfigMap — Pièges courants

Variables d'environnement

PAS de rechargement automatique.
Modifier le ConfigMap ne suffit pas !

Il faut redémarrer les pods :

```
kubectl rollout restart deploy/api
```

Volume mount (fichiers)

Mise à jour automatique (~1-2 min).
Mais l'application doit relire le fichier !

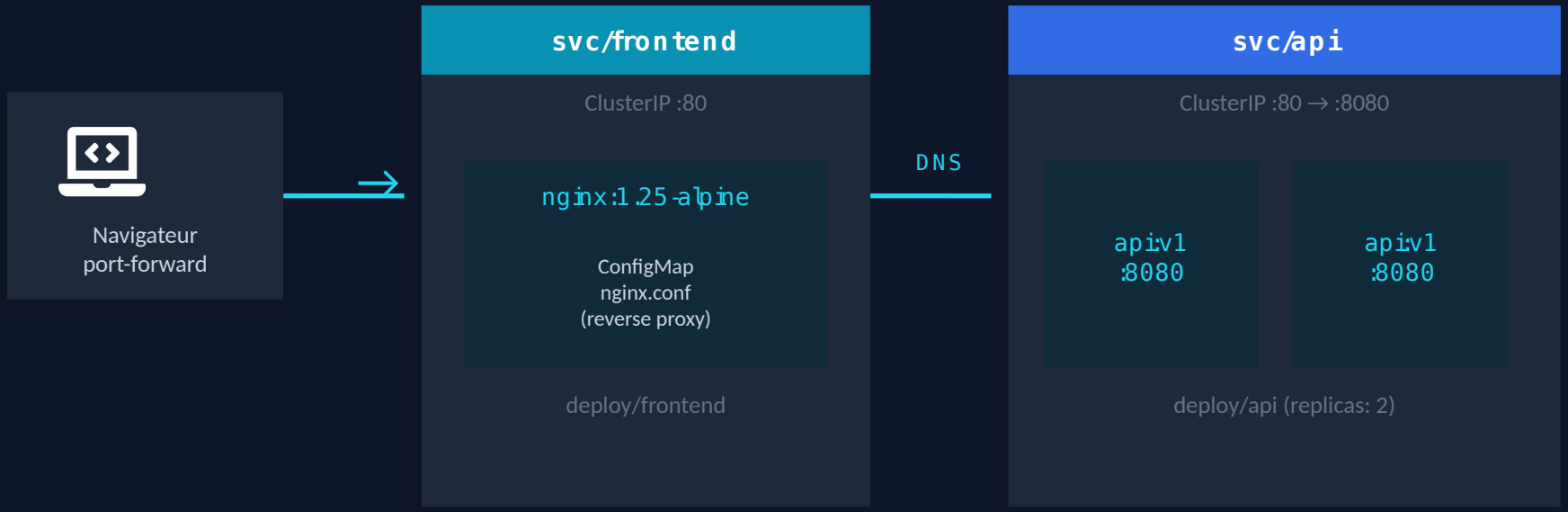
Nginx par exemple ne relit pas tout seul.
Il faut aussi restart le pod.

ConfigMap vs Secret

`ConfigMap`	Données non-sensibles (ports, URLs, feature flags)
`Secret`	Données sensibles (passwords, API keys, tokens)
`Même API`	Mais les Secrets sont encodés en base64 (pas chiffrés !)

Ce qu'on va construire

Architecture du TP



Namespace : exercices

TRAVAUX PRATIQUES

Application multi-services

1. Déployer l'API et créer son Service ClusterIP
2. Tester le DNS interne (pod debug busybox)
3. Configurer le reverse proxy nginx (ConfigMap)
4. Déployer le frontend et son Service
5. Tester la communication frontend → API
6. Rolling update vers v2 + Rollback vers v1

Durée : 1h10 | Repo : k8s-training/labs/session-02-services/

Récap & prochaine session

Ce qu'on a appris

- Deployment → ReplicaSet → Pods
- Rolling updates sans downtime
- Rollback instantané
- Services : ClusterIP, NodePort, LB
- DNS interne pour le service discovery
- ConfigMap pour externaliser la config

Mini-défi (entre les sessions)

Ajouter un ConfigMap pour externaliser le port et l'URL de l'API dans le frontend.

Bonus : tester sur kind en local !

Session 3 — Ingress et Traefik

Exposer vos services au monde extérieur avec un vrai nom de domaine.
Fini le port-forward !